

文献和相关内容分析

近年文献总体情况

总体上，近三年内的相关活动对于特定的 Solidity 实现的差分测试相对较少，相关的研究大多数着重于：

- 对于 Solidity 官方编译器系统下的测试和错误调查；
- 对于常见流行语言，例如 Rust, Kotlin 等的差分测试。

相关文献

- [he2024bugs] *Towards Understanding the Bugs in Solidity Compiler* (ISSTA 2024) – 对 solc 编译器的实证分析，一定程度上提供了一系列常见于 Solidity 编译器的可能问题类型。
- [zhang2024syntax] *Syntax-Aware Mutation for Testing the Solidity Compiler* (LNCS 2024) – 基于 MR 的测试生成，更加侧重于语法，即相对而言侧重于编译器和编译分析。
- [liu2024deep] *Differential testing solidity compiler through deep contract manipulation and mutation* (Software Quality Journal 2024) – 同样是基于 MR 的 Solidity 编译器差分测试，但是同样更多着重于 EVM 本身。
- [sun2024rustlantis] *Rustlantis: Randomized Differential Testing of the Rust Compiler* (PACMPL/OOPSLA 2024) – 差分测试，但是针对 Rust 语言。
- [zhang2024kotlin] *Evolutionary Generative Fuzzing for Differential Testing of the Kotlin Compiler* (FSE Companion 2024) – 基于模糊测试的 Kotlin 差分测试。
- [liu2023updates] *An Empirical Study of Impact of Solidity Compiler Updates on Vulnerabilities* (PerCom Workshops 2023) – 偏上层，主要强调了编译器版本对安全性的隐式影响。这一类问题也是编译器行为差异的一种。
- [flatt2023rustsmith] *RustSmith: Random Differential Compiler Testing for Rust* (ISSTA 2023) – 同上，针对 Rust 的差分测试。
- [sun2022cpp] *Detecting C++ Compiler Front-End Bugs via Grammar Mutation and Differential Testing* (IEEE Transactions on Reliability 2022) – 针对语法的 MR 测试，重点在于 C++ 编译器前端。在 Solang 中目前观察这类前端的软缺陷较多，但是实际引起的行为问题较少。
- [schafer2022jit] *Interpreter-guided differential JIT compiler unit testing* (PLDI 2022) – 通过解释器实现的 JIT 编译器差分测试，在以太坊这种相对封闭的环境中可能不太好执行，需要进一步查看相关的开源运行时。
- [johnson2022solcmc] *SolCMC: Solidity Compiler's Model Checker* (LNCS 2022) – 关于 solc 的模型，可能有一定指导作用。

文献分类

A. 直接针对 Solidity 的测试和研究

该类文献基本都着重于 `solc` 这一以太坊标准，由于 `solang` 相对使用人数和社区活跃度较低，所以相关研究较少。

B. 通用编译器测试

该类文献基本是早期文献，以 2020 年以前为主，更多的是起到一定的思路指导。其中基于 Fuzz 和 Mutation 的相对比较易于实施（考虑到 Solidity 基础设施特色）。部分基于实际二进制的结果在当前基础设施下比较困难。

C. 智能合约语义验证和分析

相对复杂，主要涉及到 Solidity 的合约语义，其中涉及到二进制分析的需要基于统一平台（例如以太坊 EVM）。